

Módulo 2: Algoritmos y Costo Computacional

Luca Sioni

r Sys.Date()

Contents

1. Introducción	1
2. Ejercicio de Inicialización (DNI)	1
3. La “Penitencia de Newton”: Comparativa de Algoritmos	1
3.1 Método 1: Bucle ‘For’ (Fuerza Bruta)	1
3.2 Método 2: Fórmula de Gauss (Analítico)	2
4. Análisis de Resultados	2
5. Conclusión	2

1. Introducción

En este informe se analiza la eficiencia de diferentes algoritmos programados en R-Cran. El objetivo principal es comprender que no solo importa llegar al resultado correcto, sino también la eficiencia (tiempo de ejecución) con la que el computador procesa la información.

2. Ejercicio de Inicialización (DNI)

Como paso previo, definimos variables basadas en las últimas cifras del DNI para personalizar los vectores de prueba.

```
{r} # Últimas 3 cifras del DNI dni_val <- 945 # Creación de vector secuencia secuencia_dni
<- (1:dni_val) head(secuencia_dni)
```

3. La “Penitencia de Newton”: Comparativa de Algoritmos

En este experimento comparamos dos formas de sumar los primeros n números naturales.

3.1 Método 1: Bucle ‘For’ (Fuerza Bruta)

Este método suma uno por uno los elementos. Es intuitivo pero consume más recursos a medida que n crece.

```
{r} n <- 1e7 # Definimos 10 millones de iteraciones system.time({ suma_iterativa <-
0 for (i in 1:n) { suma_iterativa <- suma_iterativa + i } }) cat("Resultado Suma
Iterativa:", suma_iterativa)
```

3.2 Método 2: Fórmula de Gauss (Analítico)

Utilizamos la propiedad matemática $\frac{n(n+1)}{2}$. Este método es instantáneo independientemente del tamaño de n .

```
{r} system.time({ suma_analitica <- (n * (n + 1)) / 2 }) cat("Resultado Suma Analítica:",  
suma_analitica)
```

4. Análisis de Resultados

Como se observa en los tiempos de ejecución (**elapsed time**), el método analítico es significativamente más rápido que el bucle iterativo.

- **Bucle For:** El procesador debe realizar n operaciones de suma y asignación.
- **Fórmula de Gauss:** El procesador realiza solo tres operaciones básicas (una suma, una multiplicación y una división), sin importar el valor de n .

5. Conclusión

La programación eficiente requiere conocer las herramientas matemáticas y las funciones nativas del lenguaje (como `sum()` o `seq()` en R) que suelen estar optimizadas en C++ para reducir el costo computacional.